total, or highest ratio first), to a correct but unfeasible algorithm (the brute force solution enumerating all the possible combinations) to, finally, a smart solution which would organize the computation in a more efficient fashion.

The next step, as important or even more, brought us to thinking out of the box to simplify our problem, by removing some constraint, and thus finding an easier algorithm and a better solution. This is actually another golden rule: always study your requirements thoroughly, question them and, when possible, try to remove them if that brings you a solution that is at least as valuable, or a slightly less valuable one that can be reached with much lower cost. Of course, in this process, other concerns (like laws and security, at all levels) must be taken into consideration, so some constraint can’t be removed.

In the process of describing algorithms, as we explained in the previous section, we would next describe our solution in details and provide implementation guidelines.

We will skip these steps for the 0-1 knapsack dynamic programming algorithm, both because it’s an algorithm, not a data structure, and it is vastly described in literature. Not to mention that in this chapter its purpose was just illustrating both:

- how important it is to avoid bad choices for the algorithms and data structures we use;
- the process we will follow in the next chapters when it comes to introduce a problem and its solutions.